

Kế hoạch Project 18 ngày: Financial Transaction Data Lakehouse

Phiên bản nén 18 ngày. Giữ nguyên độ chắc của phần lõi (data modeling, data quality, quarantine, Airflow) vì đây là phần nhà tuyển dụng Junior DE soi kỹ nhất. Tận dụng nền Python/Pandas/SQL sẵn có để rút gọn giai đoạn đầu. Spark/Delta có lại còn tối thiểu (1 job PySpark + Delta tables — đủ để giữ chữ "Lakehouse"). Streaming + benchmark chuyển sang "Future Work".

Cảnh báo quan trọng: Lịch này chỉ có 2 ngày đệm (Ngày 14 và Ngày 18). Airflow + Docker với người mới là rủi ro thời gian lớn nhất. Nếu trượt lịch, **lấy thời gian từ phần Spark (Ngày 15) bù vào, KHÔNG đụng vào Ngày 1-10**. Nếu buộc phải cắt thêm: bỏ patching/backfill (Ngày 14) trước, rồi đến PySpark job (Ngày 15) — nhưng **giữ Delta tables ở Ngày 16 bằng mọi giá** vì đó là thứ bảo chứng cho tên "Lakehouse".

1. Tên project

Financial Transaction Data Lakehouse for Fraud & Risk Analytics

2. Mục tiêu project

Xây dựng một hệ thống Data Engineering xử lý dữ liệu giao dịch tài chính/ngân hàng/fintech theo kiến trúc Lakehouse.

Project tập trung vào các kỹ năng:

- SQL
- Python
- Pandas
- Seaborn
- Airflow
- Spark/PySpark
- Databricks
- Delta Lake
- Data Modeling
- Data Quality
- Quarantine bad records
- Streaming
- Patching
- Performance optimization

3. Bài toán doanh nghiệp

Project mô phỏng bài toán dữ liệu trong ngân hàng, ví điện tử hoặc fintech.

Các câu hỏi cần giải:

1. Tổng số giao dịch và tổng giá trị giao dịch theo ngày là bao nhiêu?
2. Kênh giao dịch nào phổ biến nhất: mobile, web, ATM, POS, QR?
3. Khách hàng nào có hành vi giao dịch bất thường?
4. Merchant nào có tỷ lệ giao dịch thất bại hoặc rủi ro cao?
5. Có giao dịch nào có số tiền bất thường so với lịch sử khách hàng không?
6. Có khách hàng nào dùng thiết bị mới hoặc vị trí mới bất thường không?
7. Dữ liệu lỗi, duplicate hoặc đến muộn được xử lý như thế nào?
8. Có thể tạo feature table cho fraud detection hoặc credit risk không?

4. Kiến trúc tổng thể

```
CSV / API / PostgreSQL / Streaming Events
      ↓
    Bronze Layer
raw_transactions
raw_customers
raw_accounts
raw_devices
raw_exchange_rates
raw_login_events
      ↓
    Silver Layer
clean_transactions
clean_customers
clean_accounts
clean_devices
clean_exchange_rates
      ↓
    Quarantine Layer
quarantine_bad_records
quarantine_duplicate_transactions
quarantine_invalid_amount
quarantine_invalid_timestamp
      ↓
    Gold Layer
fact_transaction
dim_customer
dim_account
```

```
dim_merchant
dim_device
dim_location
dim_date
mart_daily_transaction
mart_fraud_features
mart_customer_risk
mart_merchant_risk
mart_payment_channel
```

5. Dataset và bảng dữ liệu

Có thể dùng dữ liệu public hoặc tự sinh dữ liệu giả lập.

Các bảng nguồn

Bảng	Ý nghĩa
transactions	Giao dịch chuyển tiền, thanh toán, rút tiền
customers	Hồ sơ khách hàng
accounts	Tài khoản ngân hàng hoặc ví
merchants	Đơn vị chấp nhận thanh toán
cards	Thẻ ghi nợ/thẻ tín dụng
devices	Thiết bị giao dịch
locations	Quốc gia, tỉnh, thành phố
exchange_rates	Tỷ giá
fraud_labels	Nhãn gian lận nếu có
customer_events	Login, đổi mật khẩu, thêm thiết bị, thêm người nhận

Các cột quan trọng trong bảng transactions

```
transaction_id
customer_id
account_id
merchant_id
device_id
amount
currency
transaction_type
channel
transaction_time
country
city
status
is_fraud
```

6. Công nghệ sử dụng

Thành phần	Công nghệ
Database	PostgreSQL hoặc SQLite
Batch processing	Python, Pandas
Workflow orchestration	Airflow
Big data processing	Spark / PySpark
Lakehouse	Databricks Community hoặc local Delta Lake
Storage	CSV, JSON, Parquet, Delta
Analytics	SQL, Spark SQL
Visualization	Seaborn, Notebook
Data quality	SQL checks + Python validation
Notification	Email hoặc Telegram giả lập
Version control	GitHub

7. Lịch trình chi tiết 18 ngày

Chia 4 giai đoạn: Bronze (4 ngày) → Lõi modeling/DQ/Gold (6 ngày) → Airflow (4 ngày) → Spark/Delta + đóng gói (4 ngày). Hai điểm buffer: Ngày 14 và Ngày 18.

Giai đoạn 1 — Bronze + Analytics (Ngày 1-4)

Vùng mạnh sẵn có (Python, Pandas, Seaborn, SQL). Nén mạnh, mỗi ngày gộp 1-2 việc cũ. Đừng sa đà làm dữ liệu giả lập quá đẹp — sinh nhanh rồi đi tiếp.

Ngày	Việc cần làm	Output
Ngày 1	Chốt scope fraud/risk. Tạo repo + cấu trúc folder + README nhập. Thiết kế bảng nguồn (transactions, customers, accounts, merchants, devices)	README.md, docs/project_scope.md, sql/source_schema.sql
Ngày 2	Viết script đọc CSV + API giả lập (tỷ giá/merchant) bằng Python/Pandas, có logging	src/extract/extract_csv.py, extract_api.py
Ngày 3	Load raw vào PostgreSQL hoặc Parquet Bronze + metadata (ingestion_time, source_system, batch_id, file_name). EDA nhanh bằng Pandas/Seaborn	data/bronze/, notebooks/eda_finance.ipynb
Ngày 4	SQL phân tích cơ bản: tổng giao dịch/ngày, theo kênh, top merchant, giao dịch lớn bất thường	sql/basic_analytics.sql

Kiến thức cần ôn nhanh

Nhóm	Cần học
SQL	JOIN, GROUP BY, CTE, Window Function
Python	requests, logging, try/except, config .env
Pandas	read_csv, merge, groupby, datetime, missing values
Seaborn	histogram, boxplot, heatmap, lineplot

Checklist hết Giai đoạn 1

- Dữ liệu raw đã vào Bronze + metadata.
- Có notebook EDA + SQL phân tích cơ bản.
- Có logging khi đọc/load.
- Cấu trúc repo rõ ràng.

Giai đoạn 2 — Silver, Data Quality, Star Schema, Gold (Ngày 5-10) — TRÁI TIM PROJECT

Phần được soi kỹ nhất trong phỏng vấn Junior DE. **Không nên thêm phần này.** `mart_fraud_features` là chỗ tỏa sáng nhờ nền ML (z-score, velocity features).

Ngày	Việc cần làm	Output
Ngày 5	Xác định grain (1 dòng fact = 1 transaction) + thiết kế Star Schema: fact_transaction, dim_customer, dim_account, dim_merchant, dim_device, dim_date	docs/grain_design.md, sql/star_schema.sql
Ngày 6	Transform Bronze → Silver: ép kiểu amount/timestamp, chuẩn hóa currency, channel, transaction_type	src/transform/bronze_to_silver.py
Ngày 7	Viết data quality checks (null, duplicate, amount âm, timestamp sai, FK không tồn tại)	src/quality/quality_checks.py
Ngày 8	Đưa bad records vào quarantine, lưu lý do lỗi	data/quarantine/
Ngày 9	Build Gold fact/dim tables	src/transform/build_gold.py
Ngày 10	Viết 4 mart cốt lõi *: daily_transaction, customer_risk, merchant_risk, fraud_features	sql/marts/

Rule data quality nên có

Rule	Ý nghĩa
transaction_id không null	Mỗi giao dịch phải có ID
Không duplicate transaction_id	Tránh tính trùng tiền
amount > 0	Giao dịch không được âm
transaction_time hợp lệ	Không được ở tương lai quá xa
customer_id tồn tại	Tránh giao dịch mớ côi
account_id tồn tại	Đảm bảo FK đúng
currency hợp lệ	VND, USD, EUR
channel hợp lệ	mobile, web, atm, pos, qr

device_id hợp lệ	Phục vụ phân tích rủi ro thiết bị
country/city không bất thường	Phục vụ location risk

Gold marts nên có

Nếu kết thời gian, ưu tiên 4 mart đánh dấu ✱ trước. Ba mart còn lại làm khi dư thời gian.

Mart	Mục đích
mart_daily_transaction ✱	Tổng số giao dịch, tổng tiền theo ngày
mart_channel_performance	Phân tích mobile/web/ATM/POS/QR
mart_customer_risk ✱	Khách hàng có hành vi bất thường
mart_merchant_risk ✱	Merchant có thất bại/rủi ro cao
mart_fraud_features ✱	Feature table cho fraud detection
mart_velocity_features	Số giao dịch trong 1h/24h gần nhất
mart_location_risk	Giao dịch khác quốc gia/tỉnh bất thường

Checklist hết Giai đoạn 2

- Có Silver clean data + quarantine bad records.
- Có Star Schema + Gold fact/dim.
- Có 4 mart cốt lõi ✱.
- Có quality report.

Giai đoạn 3 — Airflow, Error Handling, Patching (Ngày 11-14)

Rủi ro thời gian lớn nhất. Dùng image Airflow chính thức, đừng tự build. Ngày 14 là buffer thật — đừng lấp kín.

Ngày	Việc cần làm	Output
Ngày 11	Cài Airflow bằng Docker Compose (image chính thức)	docker-compose.yml
Ngày 12	Tạo DAG lõi: extract → load bronze → validate → transform silver → build gold marts → send report	dags/finance_batch_pipeline.py
Ngày 13	Thêm retry/timeout/logging/notification giả lập + inject bad data và kiểm tra quarantine không làm fail toàn pipeline (điểm nhấn phỏng vấn rất hay)	DAG hoàn chỉnh, src/utlis/notify.py, docs/bad_data_handling.md
Ngày 14	BUFFER. Nếu dư thời gian: demo patching/backfill nhẹ (rerun 1 partition) + viết docs Airflow. Nếu thiếu: dùng để bù việc trễ	docs/patching_demo.md, docs/airflow_setup.md

Airflow DAG đề xuất

```
start
↓
extract_transactions
↓
load_bronze
↓
validate_bronze
↓
transform_silver
↓
validate_silver
↓
build_gold_marts
↓
send_quality_report
↓
end
```

Checklist hết Giai đoạn 3

- Airflow DAG chạy end-to-end.
- Task fail thì retry, có log rõ lỗi ở bước nào.
- Bad data được đưa vào quarantine, không làm fail pipeline.
- Có quality report.

Giai đoạn 4 — Spark/Delta demo + Đóng gói (Ngày 15-18)

Demo nhỏ nhưng hiểu rõ, đủ để ghi CV trung thực. **Ngày 16 (Delta tables) là must-keep** — bỏ cái này thì không còn gọi là "Lakehouse" nữa. **KHÔNG** làm streaming/benchmark.

Ngày	Việc cần làm	Output
Ngày 15	Convert một job Pandas → PySpark (bronze_to_silver). Hiểu rõ tại sao, không copy mù. <i>(Hi sinh được nếu kẹt giờ)</i>	spark/bronze_to_silver_spark.py
Ngày 16	[MUST-KEEP] Tạo Bronze/Silver/Gold dưới dạng Delta table (local Delta Lake hoặc Databricks Community) + partition theo transaction_date	Delta tables
Ngày 17	README hoàn chỉnh + architecture diagram + data model diagram + demo script. Streaming/benchmark ghi vào "Future Work"	final docs
Ngày 18	BUFFER. Quay video demo / chụp screenshot cho CV-GitHub, dọn repo	demo assets

Thứ tự hi sinh nếu trượt lịch: (1) bỏ patching Ngày 14 → (2) bỏ PySpark job Ngày 15 (giữ build_gold bằng Pandas/SQL vẫn hợp lệ) → nhưng **luôn giữ Delta tables Ngày 16.**

Checklist hết Giai đoạn 4

- Có Delta Lake Bronze/Silver/Gold + partition (bắt buộc).
- Có PySpark transform (ít nhất 1 job, nếu kịp).
- Có README hoàn chỉnh + architecture/data model diagram.
- Có demo project end-to-end (video/screenshot).
- Streaming + benchmark đã ghi rõ trong "Future Work".

Streaming event mẫu (tham khảo cho Future Work)

```
{
  "transaction_id": "txn_001",
  "customer_id": "cus_123",
  "account_id": "acc_456",
  "amount": 2500000,
  "currency": "VND",
  "channel": "mobile",
  "device_id": "dev_888",
  "merchant_id": "mch_999",
  "transaction_time": "2026-06-10T10:30:00",
  "city": "Hà Nội",
  "country": "VN",
  "status": "success"
}
```

8. Business questions cần giải bằng SQL

Câu hỏi	Bảng/Mart dùng
Tổng số giao dịch theo ngày là bao nhiêu?	mart_daily_transaction
Tổng giá trị giao dịch theo channel?	mart_channel_performance
Khách hàng nào có số giao dịch tăng đột biến?	mart_velocity_features
Giao dịch nào có amount bất thường so với lịch sử khách hàng?	mart_fraud_features
Thiết bị nào dùng bởi nhiều khách hàng khác nhau?	dim_device + fact_transaction
Merchant nào có tỷ lệ giao dịch thất bại cao?	mart_merchant_risk
Tỉnh/thành nào có nhiều giao dịch rủi ro?	mart_location_risk
Có giao dịch nào xảy ra ở 2 địa điểm xa nhau trong thời gian ngắn?	mart_fraud_features
Payment channel nào có fraud rate cao nhất?	mart_channel_risk
Khách hàng nào nên bị đưa vào watchlist?	mart_customer_risk

9. Feature table cho fraud/risk

Bảng đề xuất: mart_fraud_features

Feature	Ý nghĩa
txn_amount	Số tiền giao dịch
avg_amount_7d	Trung bình giao dịch 7 ngày

amount_zscore	Mức bất thường so với lịch sử
txn_count_1h	Số giao dịch trong 1 giờ
txn_count_24h	Số giao dịch trong 24 giờ
unique_merchants_24h	Số merchant khác nhau trong 24 giờ
unique_devices_7d	Số thiết bị dùng trong 7 ngày
is_new_device	Thiết bị mới
is_new_location	Địa điểm mới
failed_txn_count_24h	Số giao dịch fail trong 24 giờ
night_transaction_flag	Giao dịch ban đêm
high_amount_flag	Giao dịch giá trị cao
cross_country_flag	Giao dịch khác quốc gia
velocity_risk_score	Điểm rủi ro theo tốc độ giao dịch

10. Cấu trúc GitHub cuối cùng

```
finance-transaction-lakehouse/
├── README.md
├── docker-compose.yml
├── requirements.txt
├── .env.example
├── data/
│   ├── raw/
│   ├── bronze/
│   ├── silver/
│   ├── gold/
│   ├── quarantine/
│   └── bad_data_samples/
├── sql/
│   ├── source_schema.sql
│   ├── star_schema.sql
│   ├── quality_checks.sql
│   ├── basic_analytics.sql
│   └── marts/
├── src/
│   ├── extract/
│   ├── load/
│   ├── transform/
│   ├── quality/
│   └── utils/
├── dags/
│   └── finance_batch_pipeline.py
├── spark/
│   ├── bronze_to_silver_spark.py
│   ├── build_gold_spark.py           # optional nếu kịp
│   ├── stream_transactions.py       # Future Work
│   └── benchmark_spark.py            # Future Work
├── notebooks/
│   └── eda_finance.ipynb
├── docs/
│   ├── architecture.md
│   ├── data_modeling.md
│   ├── data_quality_report.md
│   ├── patching_demo.md
│   ├── benchmark.md
│   └── business_questions.md
├── images/
│   ├── architecture_diagram.png
│   ├── star_schema.png
│   └── airflow_dag.png
```

11. Mô tả project để đưa vào CV

Financial Transaction Data Lakehouse for Fraud & Risk Analytics

Built an end-to-end financial transaction data lakehouse using SQL, Python, Airflow, PySpark, Databricks and Delta Lake. Designed Bronze/Silver/Gold layers, implemented data quality checks, bad record quarantine, retry/backfill workflows, and analytical marts for transaction monitoring, customer risk,

12. Mức độ ưu tiên trong 18 ngày

Thứ tự ưu tiên (cái trên cùng quan trọng nhất, cắt từ dưới lên nếu trượt lịch):

1. SQL + Python + Pandas + Bronze (Giai đoạn 1)
2. Silver + Data quality + quarantine (Giai đoạn 2)
3. Star Schema + 4 Gold marts ✱ (Giai đoạn 2)
4. Airflow DAG end-to-end (Giai đoạn 3)
5. **Delta tables** (Giai đoạn 4, Ngày 16) — must-keep để giữ tên "Lakehouse"
6. PySpark 1 job (Giai đoạn 4, Ngày 15) — hi sinh được nếu kẹt giờ
7. ~~Patching/backfill~~ → để vào buffer Ngày 14
8. ~~Streaming + benchmark~~ → Future Work

Hoàn thành tốt mục 1-4 là project đã đủ mạnh cho Junior DE. Mục 5 giữ chữ "Lakehouse" cho tên project. Mục 6 tăng độ nổi bật. Mục 7-8 không bắt buộc trong đợt này.

13. Future Work (ghi vào README)

Phần này ghi trung thực vào README — vừa không phải làm gấp, vừa cho thấy mình hiểu hệ thống nên tiến hóa ra sao (nhà tuyển dụng đánh giá cao):

- **Real-time ingestion** với Spark Structured Streaming (đọc transaction events liên tục, ghi Bronze streaming).
- **Performance benchmarking** Pandas vs Spark trên 1M/5M/10M dòng.
- **Partition tuning nâng cao** (theo channel, country) + Delta OPTIMIZE/Z-ORDER.
- **CI/CD** cho pipeline và data quality tests.

Cách viết CV trung thực: vẫn ghi được *SQL, Python, Airflow, PySpark, Delta Lake, data quality, quarantine, backfill* vì đều có dụng tới thật. Streaming/benchmark để ở Roadmap, đừng claim là đã hoàn thành.